

salesforce

Uncovered Global Secondary Indexes

April 2023

Kadir Ozdemir and Tanuj Khurana



THANK YOU



Outline



- Terminology
- Covered Global Indexes
- Storage and Performance Challenges
- Uncovered Global Indexes
- Covered Index Read Optimization for Uncovered Queries
- Case Study
- Benefits
- Next Steps



Terminology



Strongly Consistent Indexes do not change the query results

A query is **covered** by an index if the index includes all the columns of the query

A **covered/covering index** answers covered queries directly without accessing the data table

An **uncovered index** cannot answer directly any queries including covered queries



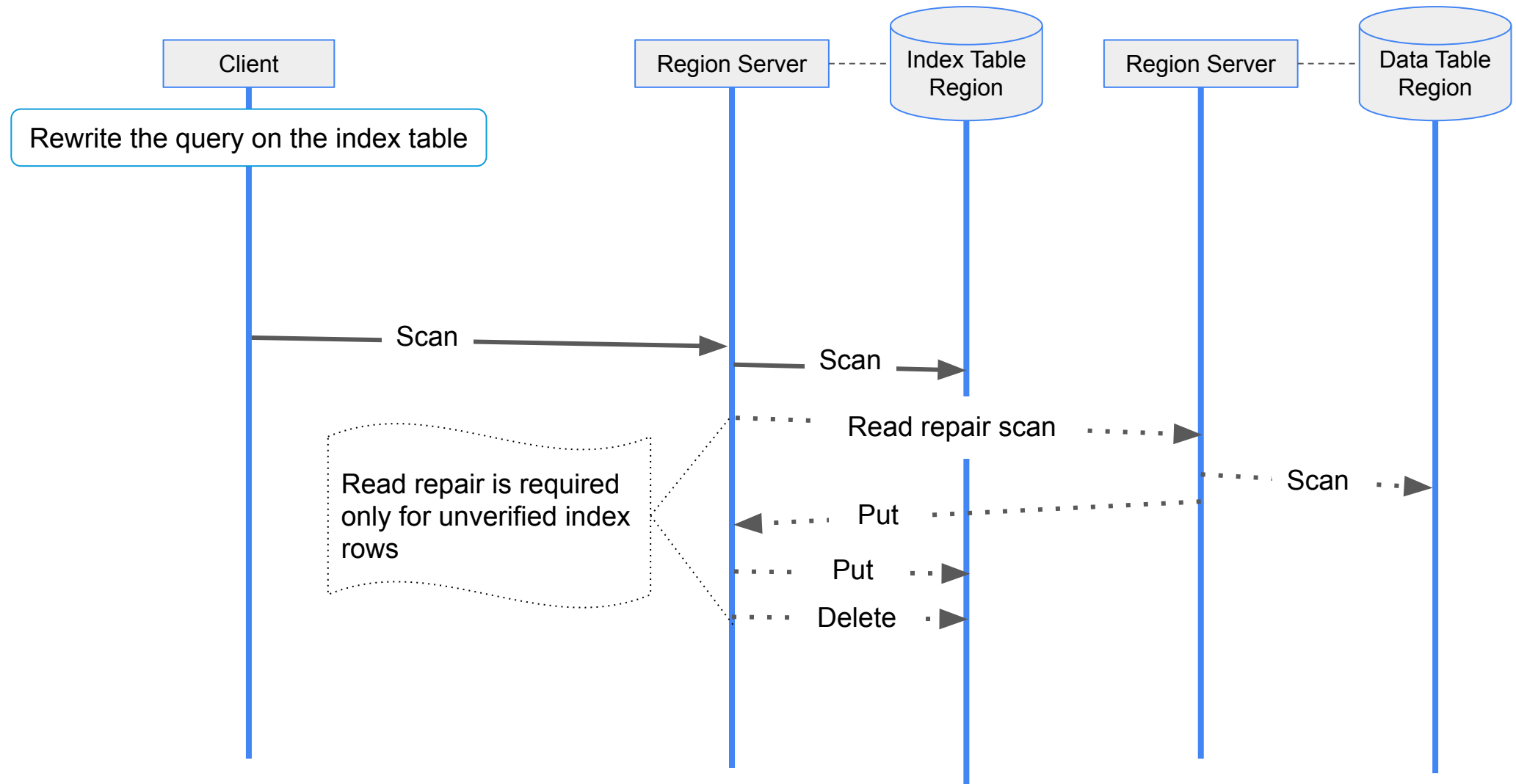
Covered Global Indexes



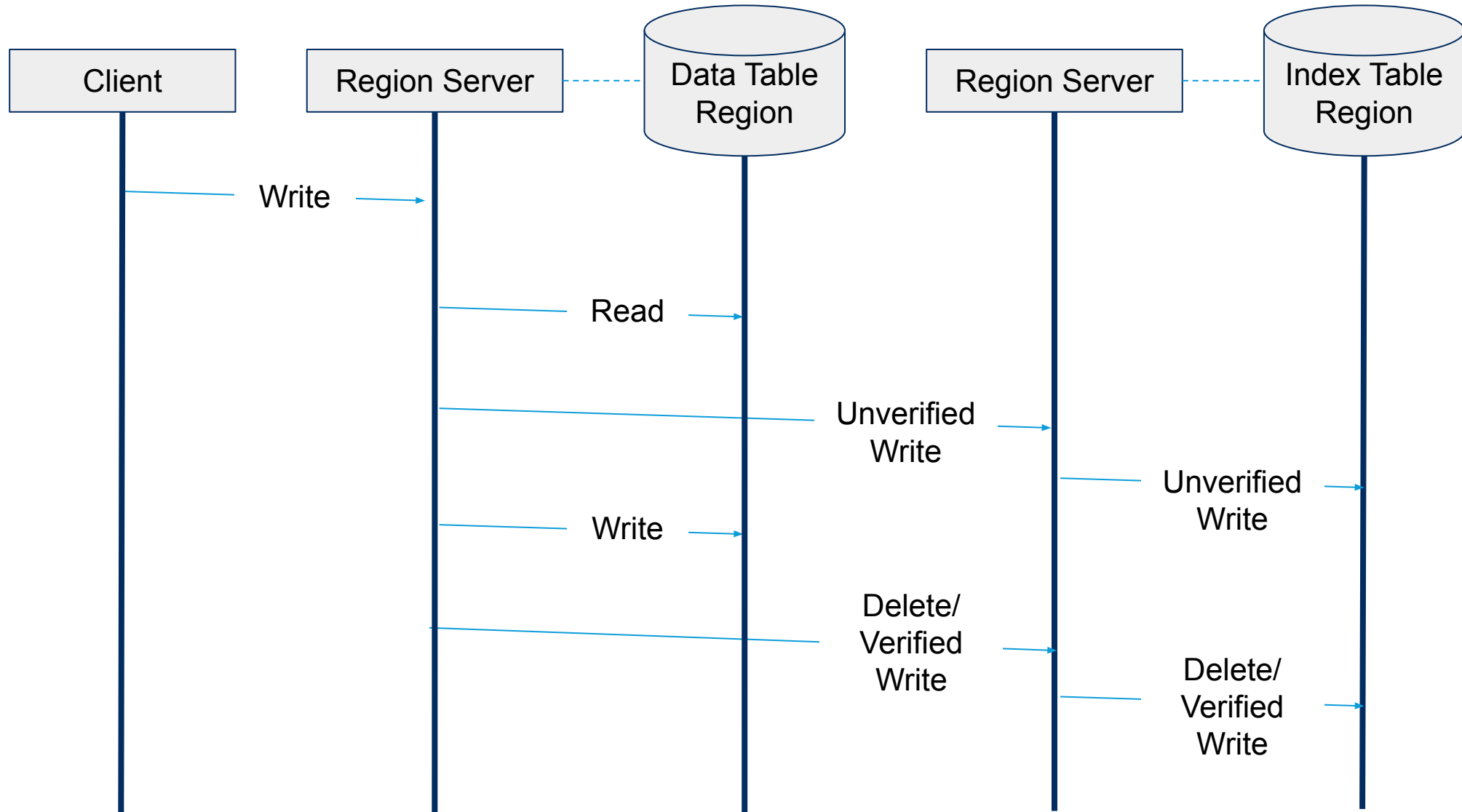
- A covered index includes
 - the primary key columns of the data table,
 - the secondary key columns for which the index is created, and
 - optionally other columns
- **CREATE INDEX** <index name> **ON** <data table name> (<secondary key columns>) **INCLUDE** (<additional columns>)
- Covered indexes
 - serve covered queries directly without accessing the data table
 - can provide the best query performance
- All indexes were covered in Phoenix



Covered Index Read Path for Covered Queries



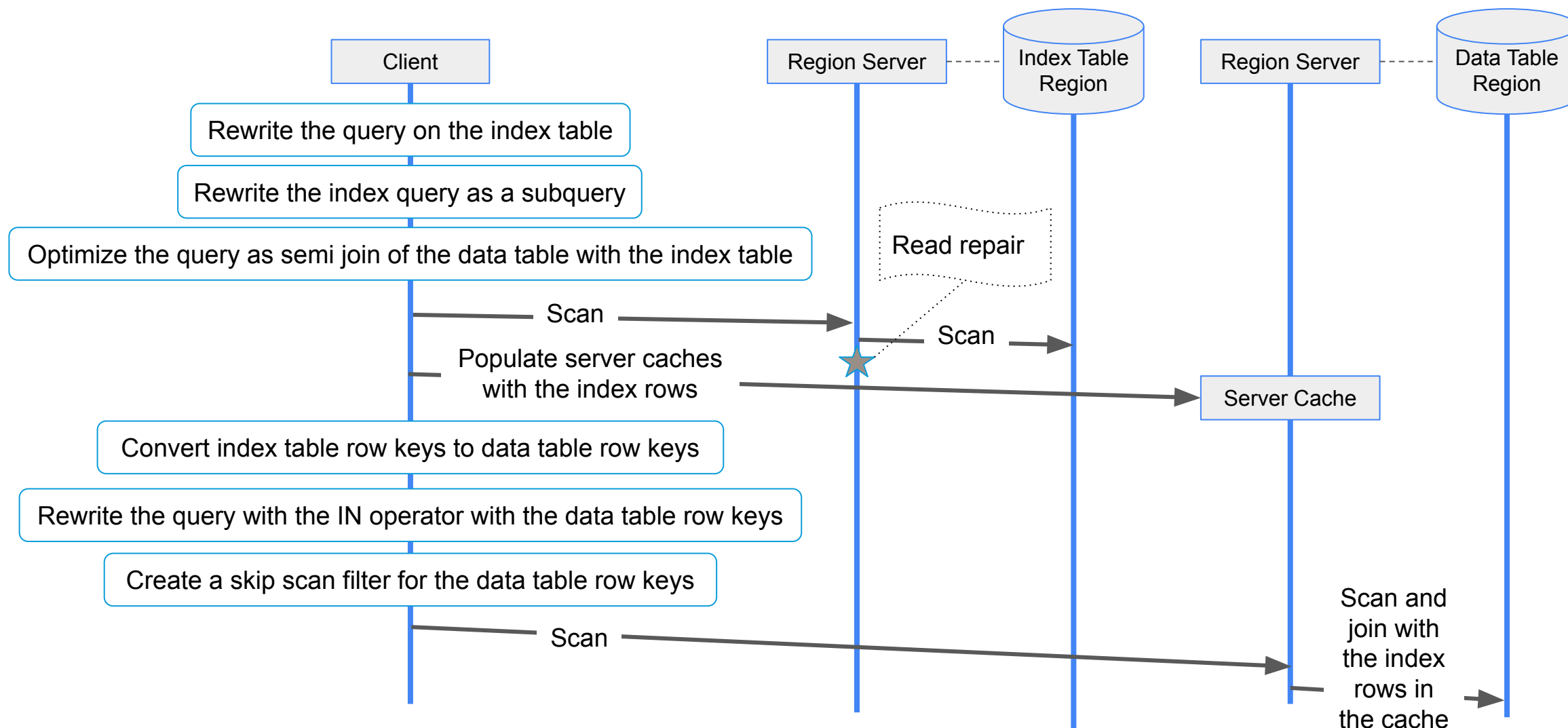
Covered Index Write Path



Covered Index Read Path for Uncovered Queries



Before



Covered Index Storage and Write Performance Challenges



- Can have large storage footprint due to included non-key columns
- Can add significant write overhead
 - one read on the data table
 - two-phase commit write on the index to update/insert an index row
 - two-phase commit write on the index to invalidate/delete an index row
- Write impact is especially visible in UPSERT SELECT and DELETE
 - Large number of rows can be updated and deleted by these statements



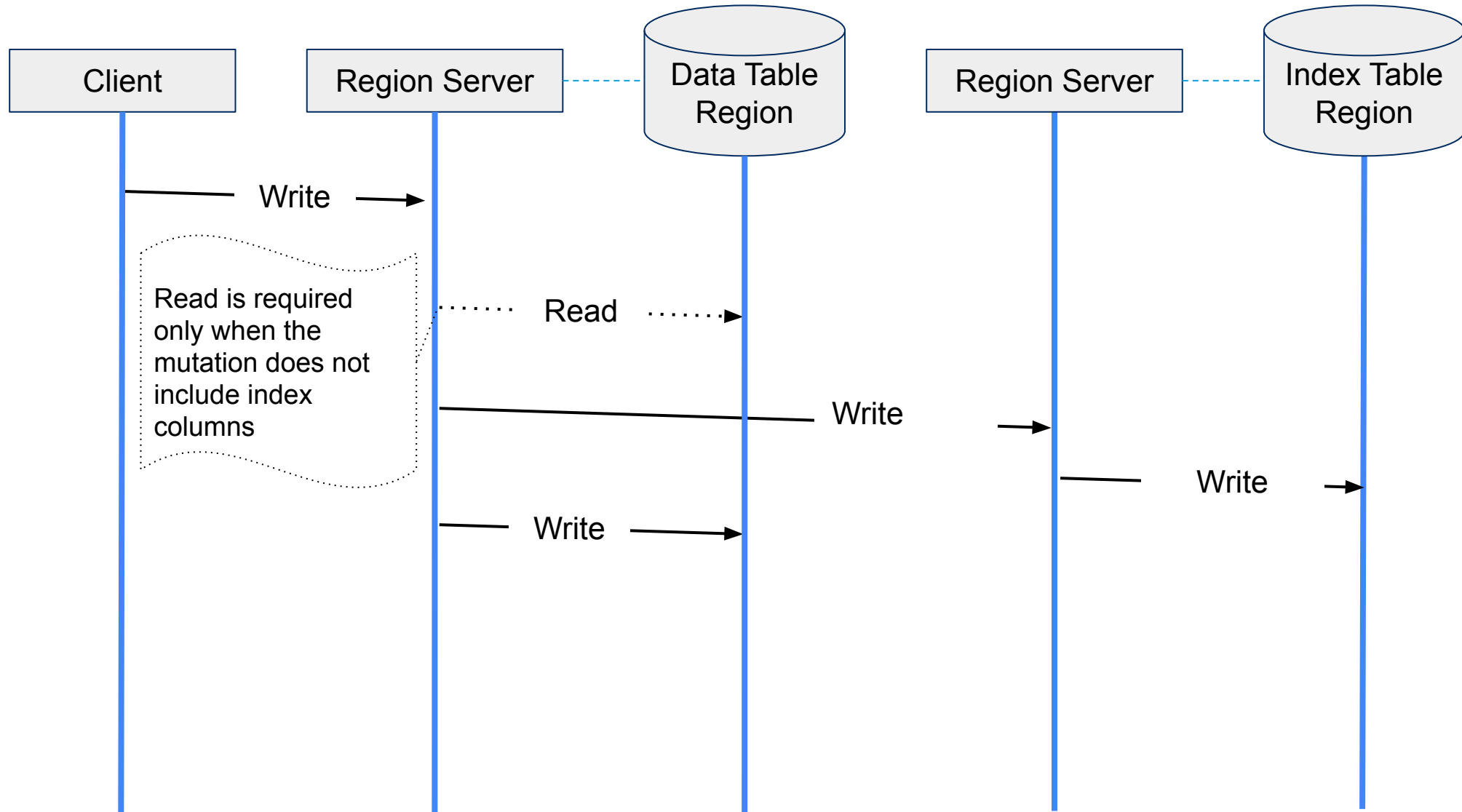
Uncovered Global Indexes



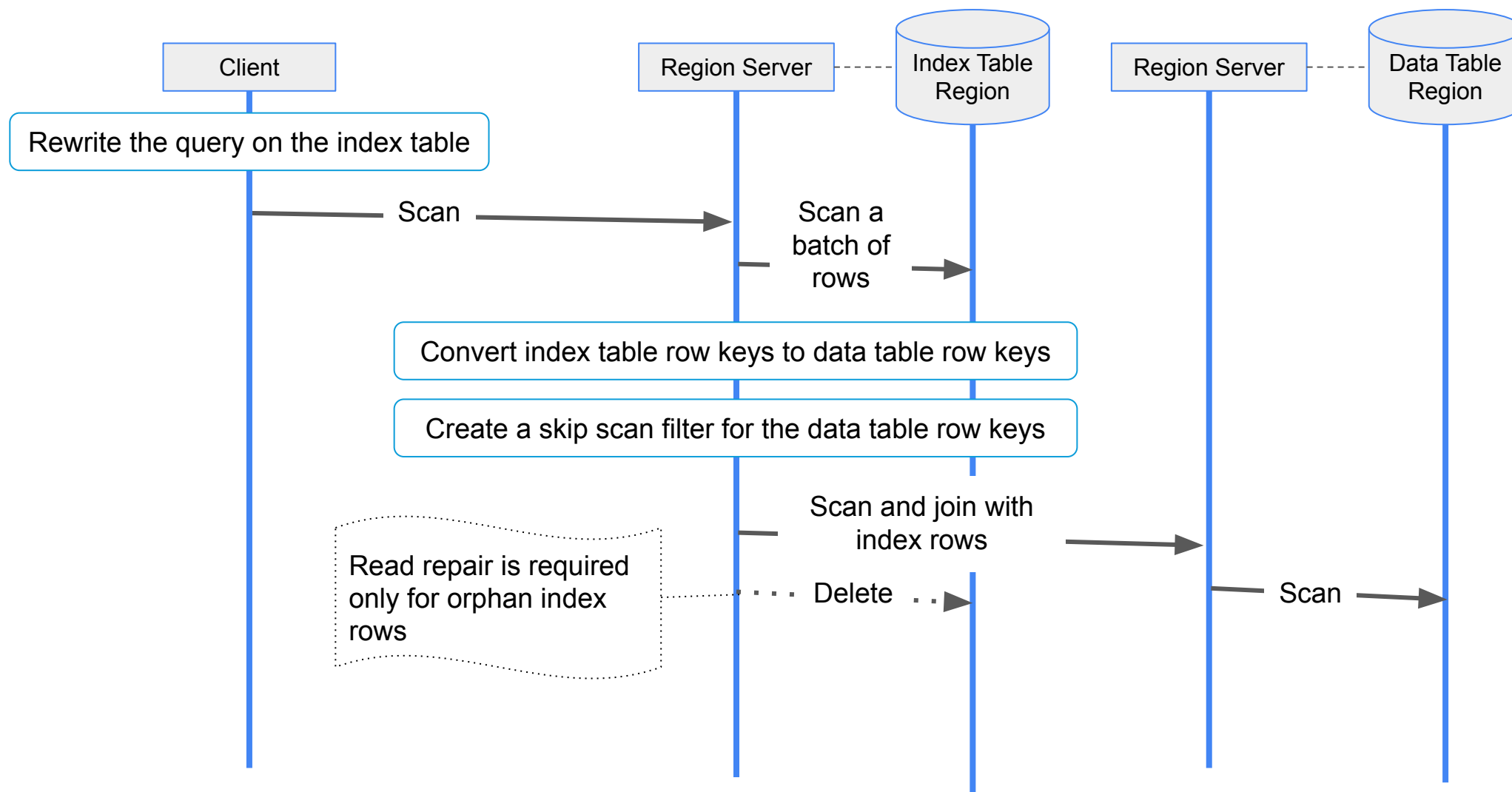
- Recently added to Phoenix
- An uncovered indexes include only
 - the primary key columns of the data table and
 - the secondary key columns for which the index is created
- **CREATE UNCOVERED INDEX** <index name> **ON** <data table name> (<secondary key columns>)
- Uncovered indexes
 - are write optimized
 - are storage optimized
 - scan both the index and data table always



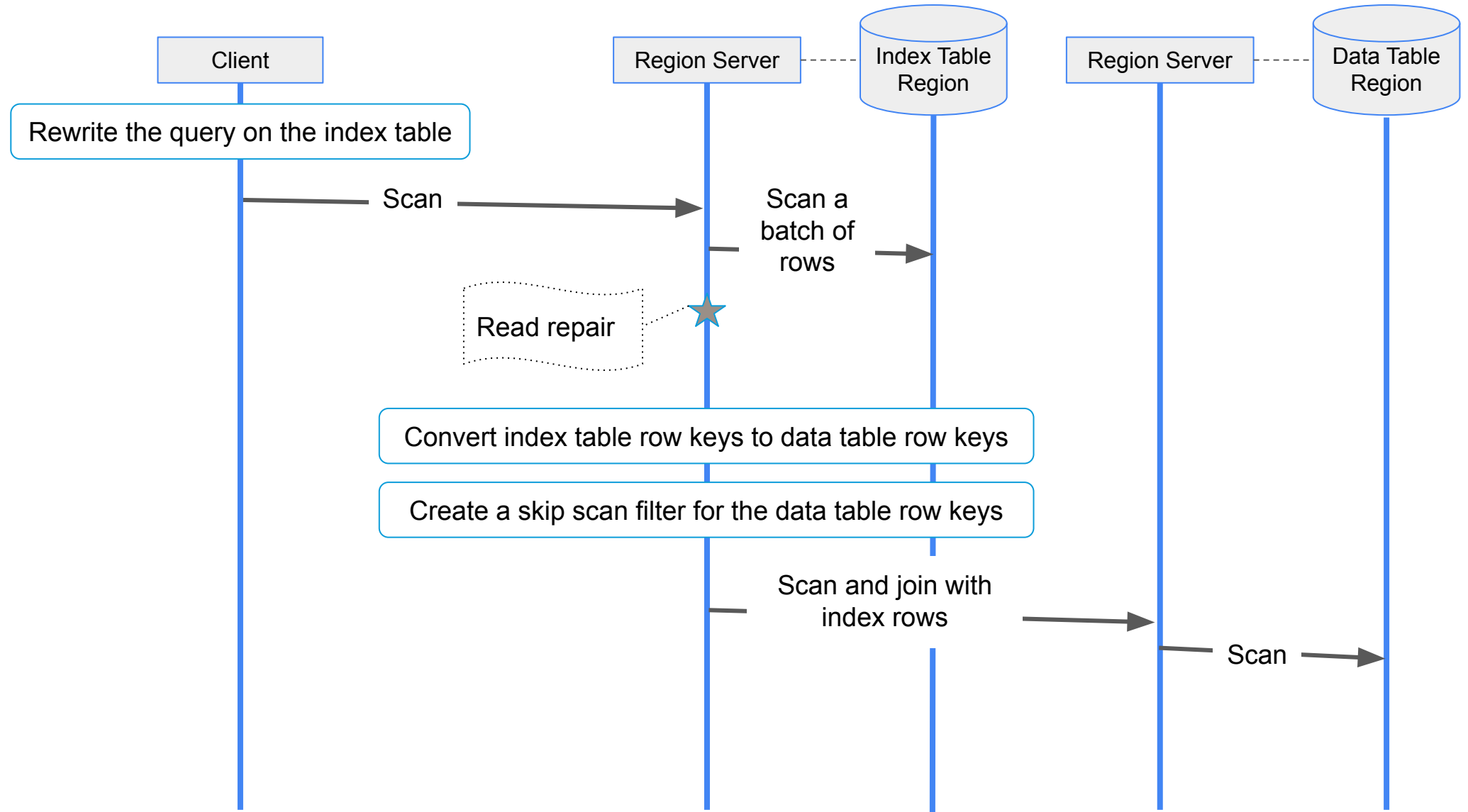
Uncovered Index Write Path



Read Path for Uncovered Indexes



Read Optimization for Covered Indexes For Uncovered Queries



Differences



	Covered Index	Uncovered Index
Write Path	• One data row read • One data row write • One two-phase-commit index row write • One two-phase-commit index row delete (when required)	• One index row write • One data row write • One data row read when required
Read Path	• One index row read • Single row read repair ◦ One data row read ◦ One index row write ◦ One index row delete (when required)	• One index row read • One data table row read • Single row read repair ◦ One index row delete
Performance	Read optimized	Write optimized
Storage	May have more storage footprint than data table does due to full index row writes	Storage optimized

Case Study

Vegacache Bajaj SMS Use-case



```
CREATE TABLE VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY (  
    CLIENT_ID VARCHAR(100) NOT NULL,  
    NAMESPACE VARCHAR(50) NOT NULL,  
    TENANT_ID VARCHAR(30) NOT NULL,  
    CACHE_KEY VARCHAR(255) NOT NULL,  
    SECONDARY_KEY VARCHAR(255),  
    CACHE_VALUE VARBINARY,  
    SIZE INTEGER,  
    CREATED_DATE DATE,  
    EXPIRY_DATE DATE,  
    CONSTRAINT PK_MUTABLE_DATA_WITH_SECONDARY_KEY PRIMARY KEY  
    (  
        CLIENT_ID,  
        NAMESPACE,  
        TENANT_ID,  
        CACHE_KEY  
    )  
);
```

```
CREATE INDEX MUTABLE_DATA_SECONDARY_KEY_INDEX ON  
VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY (  
    CLIENT_ID,  
    NAMESPACE,  
    TENANT_ID,  
    SECONDARY_KEY  
)  
INCLUDE (  
    CACHE_KEY,  
    EXPIRY_DATE  
);
```

Case Study



```
SELECT /*+ INDEX(VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY MUTABLE_DATA_SECONDARY_KEY_INDEX*/ CACHE_KEY,
CACHE_VALUE, SECONDARY_KEY FROM VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY WHERE CLIENT_ID=? and
NAMESPACE=? and TENANT_ID=? and SECONDARY_KEY >= ? and SECONDARY_KEY < ? and EXPIRY_DATE > ? ORDER BY
SECONDARY_KEY, CACHE_KEY ASC LIMIT 250
```

Size of Data Set: 1M rows

	No Index	Hint with Join Compiler	Hint with New Implementation
Compilation Time	4 ms	14772 ms	5 ms
ResultSet Processing Time	8238 ms	13618 ms	479 ms
Total Execution Time	8242 ms	28390 ms	484 ms

Speed up of **60x** compared to the old join compiler implementation

Case Study #2



Uncovered (write) optimized index VS Covered Index for Uncovered Queries

```
CREATE TABLE VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY (  
  CLIENT_ID VARCHAR(100) NOT NULL,  
  NAMESPACE VARCHAR(50) NOT NULL,  
  TENANT_ID VARCHAR(30) NOT NULL,  
  CACHE_KEY VARCHAR(255) NOT NULL,  
  SECONDARY_KEY VARCHAR(255),  
  CACHE_VALUE VARBINARY,  
  SIZE INTEGER,  
  CREATED_DATE DATE,  
  EXPIRY_DATE DATE,  
  CONSTRAINT PK_MUTABLE_DATA_WITH_SECONDARY_KEY PRIMARY KEY  
  (  
    CLIENT_ID,  
    NAMESPACE,  
    TENANT_ID,  
    CACHE_KEY  
  )  
);
```

```
CREATE UNCOVERED INDEX MUTABLE_DATA_SECONDARY_KEY_INDEX  
ON VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY (  
  CLIENT_ID,  
  NAMESPACE,  
  TENANT_ID,  
  SECONDARY_KEY  
);
```

Case Study #2



```
SELECT /*+ INDEX(VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY MUTABLE_DATA_SECONDARY_KEY_INDEX*/ CACHE_KEY,
CACHE_VALUE, SECONDARY_KEY FROM VEGACACHE.MUTABLE_DATA_WITH_SECONDARY_KEY WHERE CLIENT_ID=? and
NAMESPACE=? and TENANT_ID=? and SECONDARY_KEY >= ? and SECONDARY_KEY < ? and EXPIRY_DATE > ? ORDER BY
SECONDARY_KEY, CACHE_KEY ASC LIMIT 250
```

Size of Data Set: 1M rows

	Covered Index	Uncovered Index
Upsert time **	55055 ms	43736 ms
Compilation Time	6 ms	7 ms
ResultSet Processing Time	456 ms	501 ms
Total Execution Time	462 ms	508 ms

** UPSERT VALUES

Write Speed up of **1.2x** for uncovered index compared to **1.1x** slow down on Reads

Benefits

- Storage reduction
- Performance improvement for write intensive workloads
- Read performance optimization on covered indexes for uncovered queries
- Foundation for the Phoenix native Change Data Capture (CDC) feature
 - Use cases
 - Incremental backups
 - SeaS
 - ZoS



Next Steps

- Further performance study
- Blog/white paper
- CDC Design

salesforce

References



- [Uncovered Global Secondary Indexes](#)
- [SeaS Document Store on Phoenix](#)
- [PHOENIX-6434 Secondary Indexes on PHOENIX_ROW_TIMESTAMP\(\)](#)
- [PHOENIX-6458 Using global indexes for queries with uncovered columns](#)
- [PHOENIX-6501 Use batching when joining data table rows with uncovered global index rows](#)
- [PHOENIX-6663 Use batching when joining data table rows with uncovered local index rows](#)
- [PHOENIX-6832 Uncovered Global Secondary Indexes](#)